NETWORK TRAFFIC ANALYSIS USING WIRESHARK

GitHub Project Report

Prepared by: CHEKKA MADHUMITHALAKSHMI

Student ID: 2024065862

Branch: CYBER SECURITY

Objective

Capture and analyze network traffic to identify communication patterns, protocols, and potentially unusual connections.

Tools Used

• Wireshark
• Windows Command Prompt / ping
• PCAPNG capture format

Project Overview

This project demonstrates practical network traffic capture and analysis using Wireshark. Network packets were captured from the active network interface and examined using protocol filters and Wireshark statistics.

The analysis covered HTTP traffic, DNS queries, ICMP packets, source and destination IP addresses, endpoints, and conversations. The captured traffic was also saved as a PCAPNG file for later verification.

Project Workflow

1. Open Wireshark and identify available interfaces.
2. Select the active network adapter.
3. Start packet capture.
4. Generate traffic by opening websites.
5. Observe packet details.
6. Apply protocol filters such as HTTP, DNS, and ICMP.
7. Generate ICMP traffic using ping.
8. Review IP addresses, endpoints, and conversations.
9. Save and reopen the PCAPNG capture.
10. Summarize the observed traffic.

Key Skills Demonstrated

• Packet sniffing
• Network traffic analysis
• Protocol filtering
• DNS and ICMP analysis
• IP address and endpoint analysis
• Basic identification of unusual communication patterns

Evidence 1

Opening Wireshark and viewing available network interfaces.

Network Traffic Analysis using Wireshark

CHEKKA MADHUMITHALAKSHMI
2024065862
CYBER SECURITY

Step1: Open Wireshark

Description:
Wireshark application was opened from the Start Menu to begin network packet analysis.

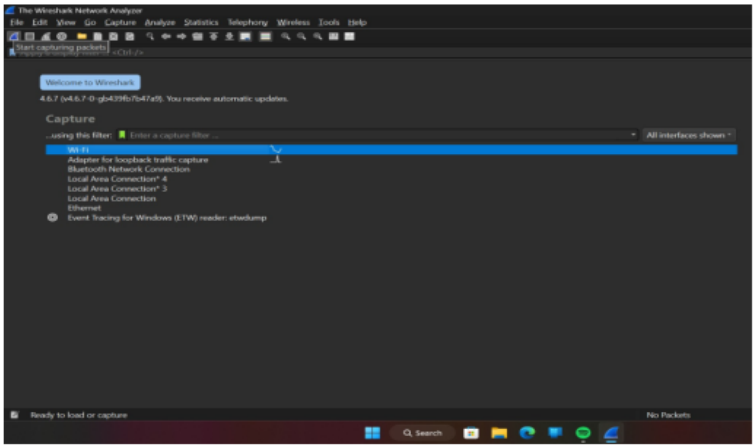


Step 2: View Available Network Interfaces

Description:
The Wireshark home screen displayed available network interfaces such as Wi-Fi and Ethernet for capturing network traffic.

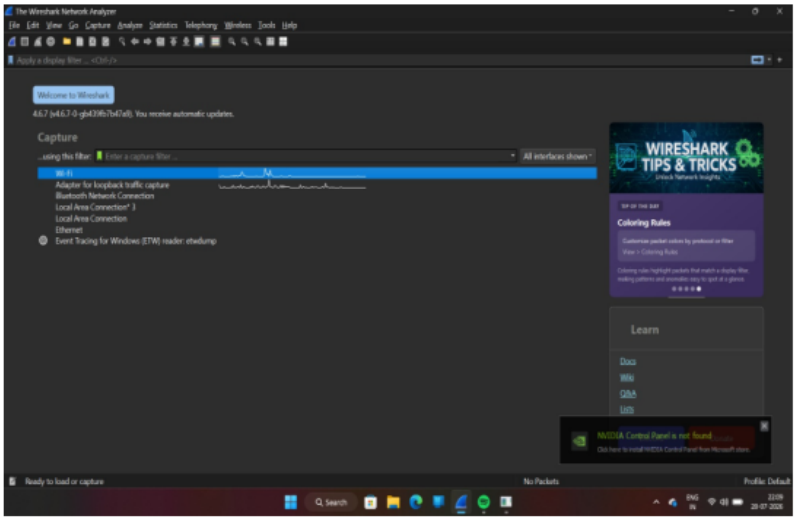
Evidence 2

Selecting the active network adapter with live traffic.



Step 3: Select Active Network Adapter

Description:
The active network interface showing live traffic was selected to capture packets from the current network connection.

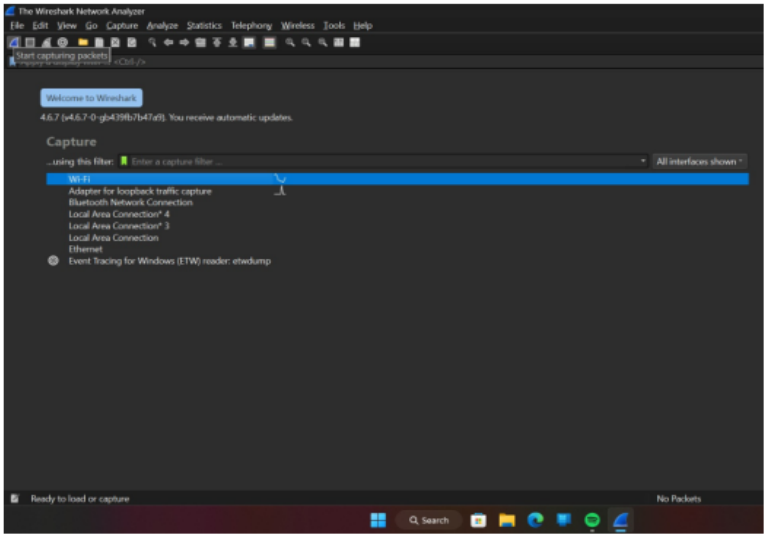


Evidence 3

Starting packet capture and generating traffic by opening websites.

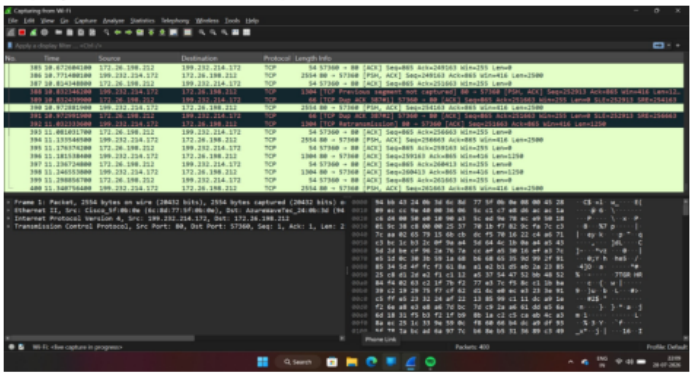
Step 4: Start Packet Capture

Description:
Packet capturing was started by clicking the blue shark-fin button, allowing Wireshark to monitor network communication.



Step 5: Generate Network Traffic and Capture Packets

Description:
Network traffic was generated by opening websites, and live packets were captured and displayed in the Packet List pane.

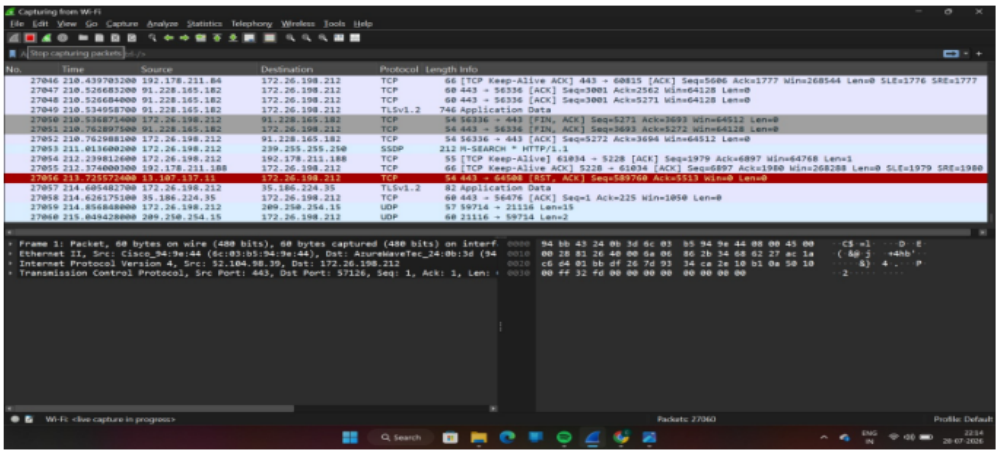


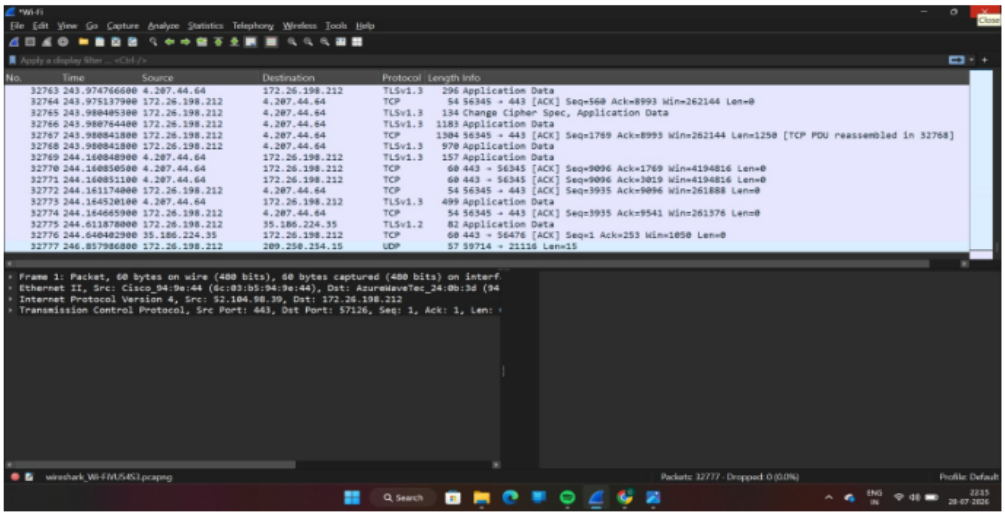
Evidence 4

Observing captured packet details including source, destination, protocol, and length.

Step 6: Observe Captured Packets

Description:
Captured packets were viewed with details such as packet number, time, source IP, destination IP, protocol, and length.



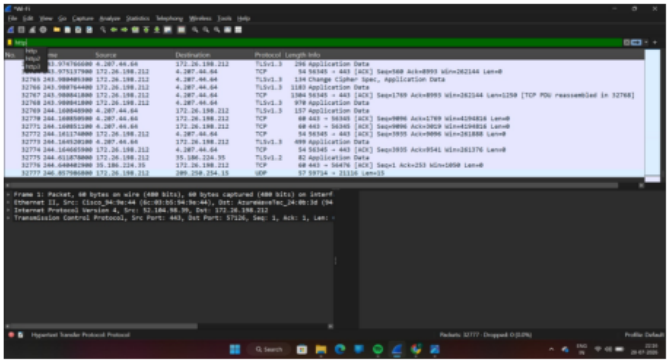


Evidence 5

Stopping the capture and applying the HTTP filter.

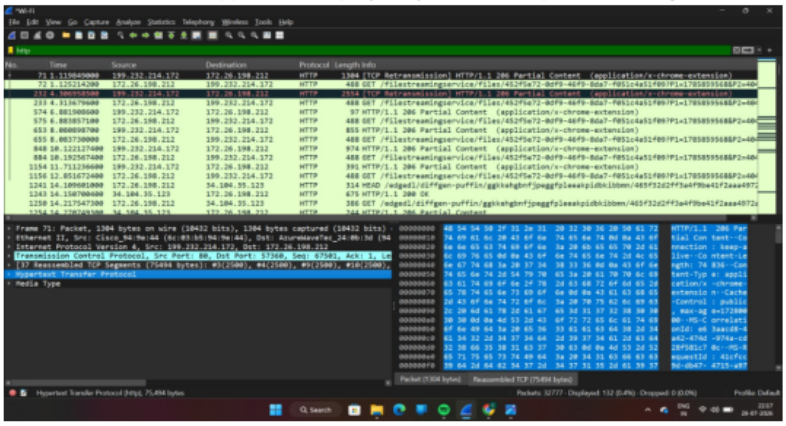
Step 7: Stop Packet Capture

Description:
The packet capture was stopped using the red stop button to analyze the collected network data.



Step 8: Apply HTTP Filter

Description:
The HTTP filter was applied to display only web traffic packets using the HTTP protocol.

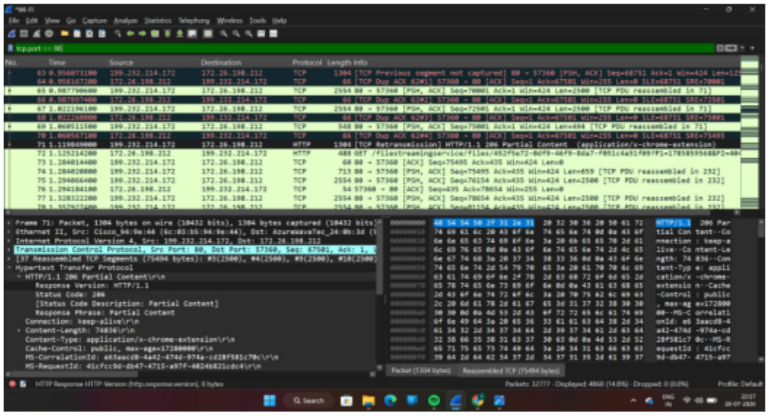


Evidence 6

Applying the TCP port 80 filter and clearing the display filter.

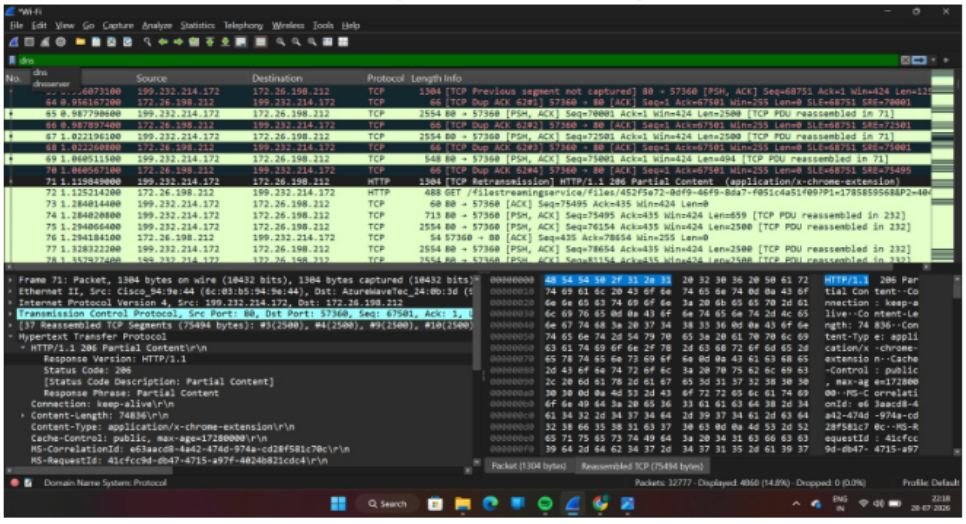
Step 9: Apply TCP Port 80 Filter

Description:
The TCP port 80 filter was used to identify HTTP communication packets when HTTP packets were not directly visible.



Step 10: Clear Display Filter

Description:
The applied filter was removed to display all captured packets again.

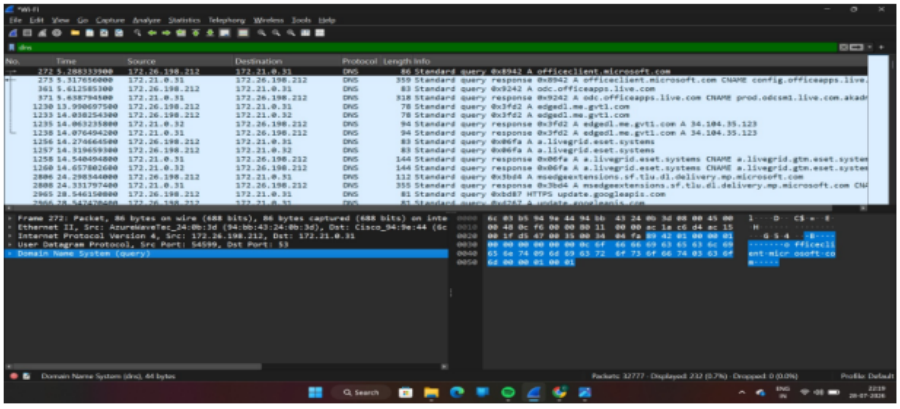


Evidence 7

Applying the DNS filter and analyzing DNS query packets.

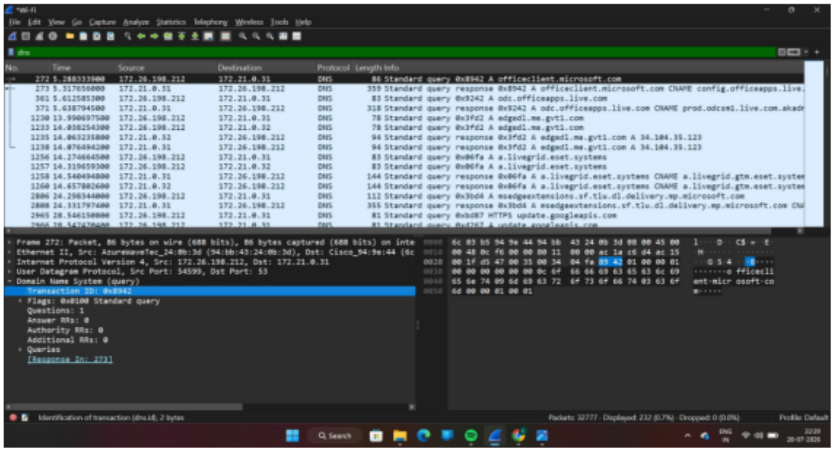
Step 11: Apply DNS Filter

Description:
The DNS filter was applied to view domain name lookup packets generated during network communication.



Step 12: Analyze DNS Query Packets

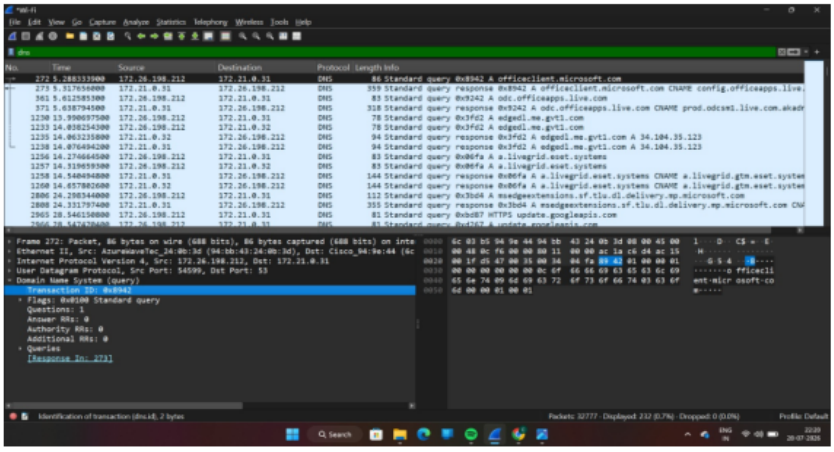
Description:
DNS packets were inspected to identify requested domain names, query types, and response details.



Evidence 8

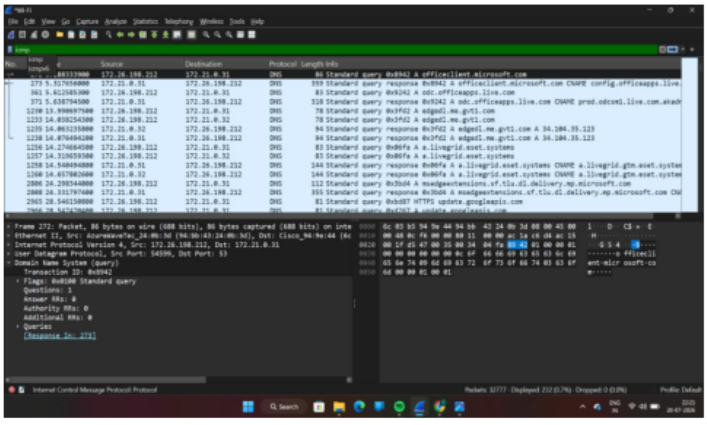
Clearing the DNS filter and applying the ICMP filter.

Step 13: Clear DNS Filter
Description:
The DNS filter was removed to continue analyzing other captured network packets.



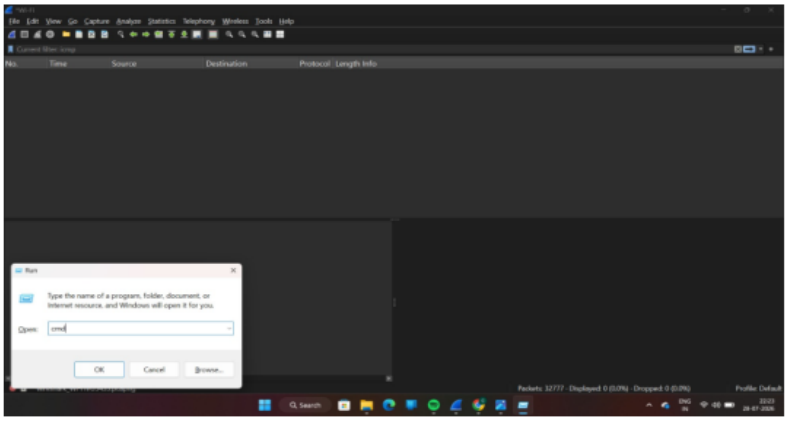
Step 14: Apply ICMP Filter

Description:
The ICMP filter was applied to display ping-related network packets.



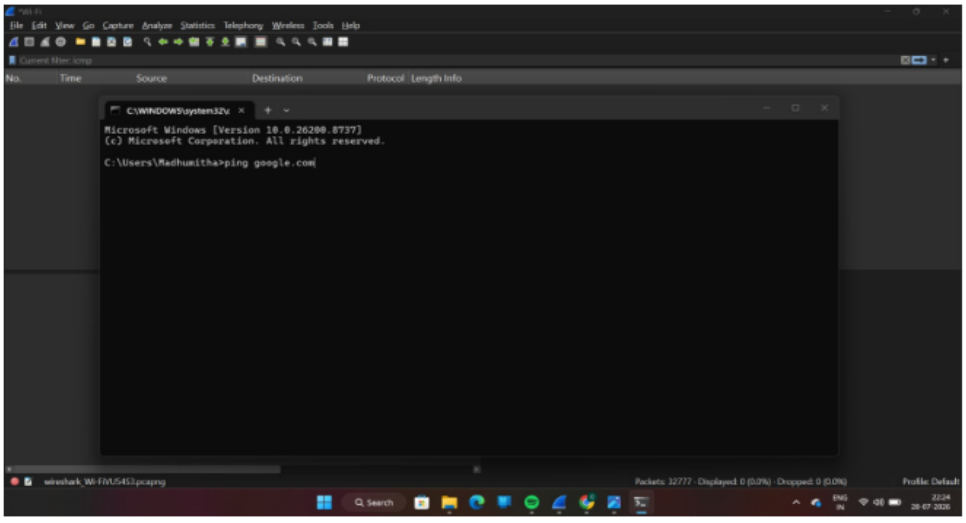
Evidence 9

Generating ICMP packets using the ping command.



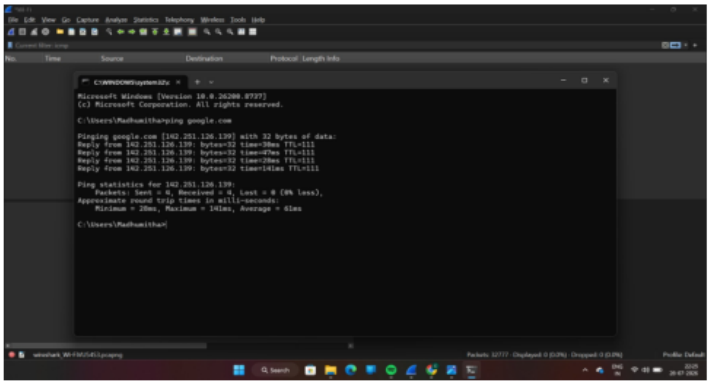
Step 15: Generate ICMP Packets Using Ping

Description:
The ping command was executed in Command Prompt to generate ICMP Echo Request and Echo Reply packets.



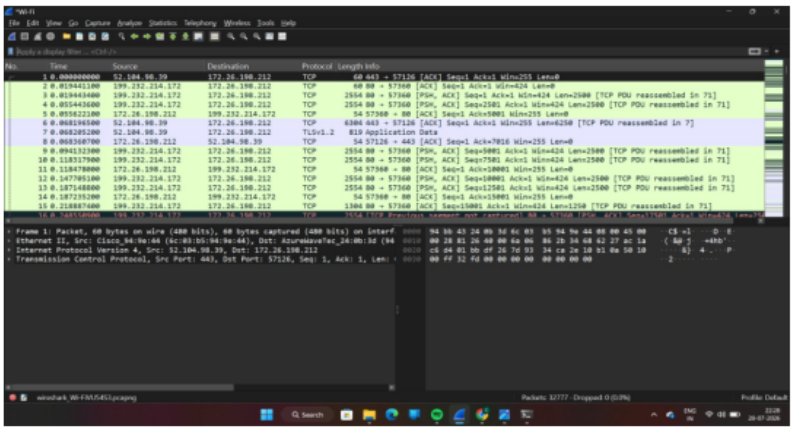
Evidence 10

Analyzing ICMP packets and identifying source and destination IP addresses.



Step 16: Analyze ICMP Packets

Description:
ICMP packets were selected to identify source and destination IP addresses involved in communication.

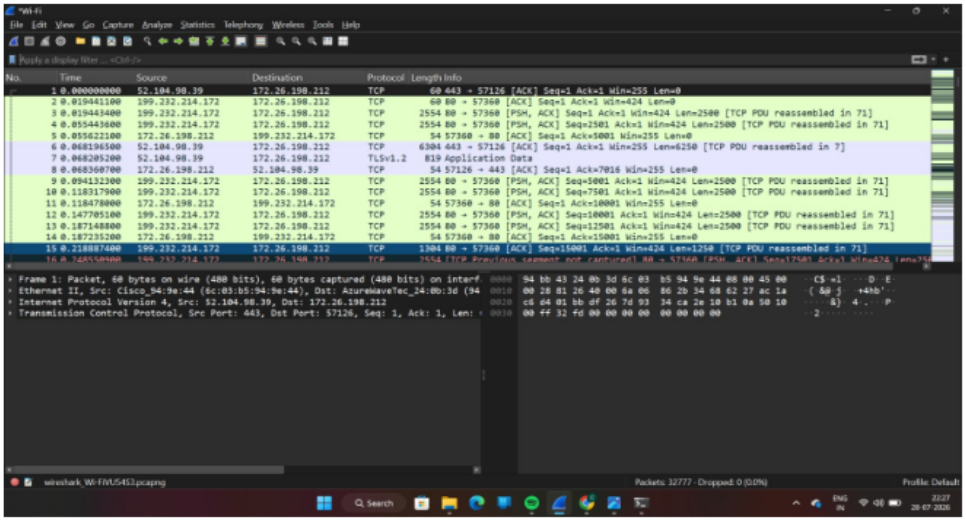


Evidence 11

Clearing the ICMP filter and reviewing the complete capture.

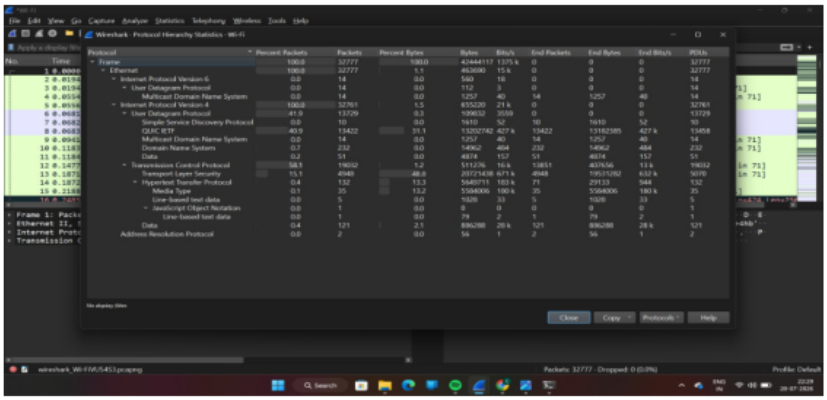
Step 17: Clear ICMP Filter

Description:
The ICMP filter was removed to return to the complete packet capture view.



Step 18: Identify Suspicious IP Addresses

Description:
Source and destination IP addresses were reviewed to identify unusual traffic patterns and unknown connections.

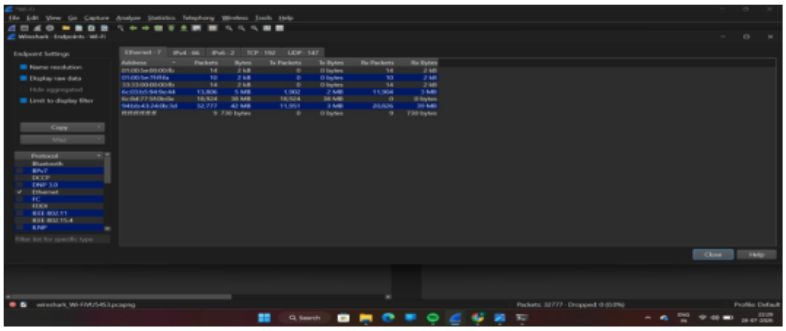


Evidence 12

Reviewing source/destination IPs and identifying potentially unusual connections.

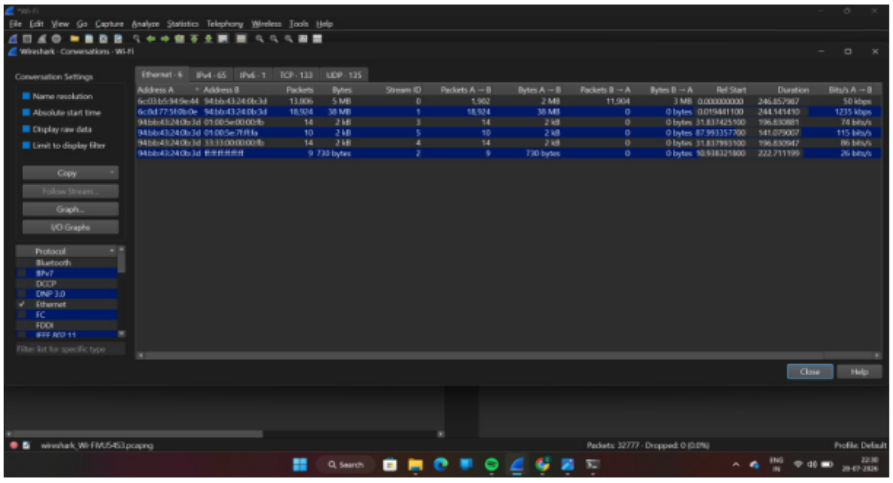
Step 19: Analyze Endpoints

Description:
The Statistics → Endpoints option was used to view all communicating IP addresses and traffic information.



Step 20: Analyze Conversations

Description:
The Statistics → Conversations option was used to analyze communication between different hosts.

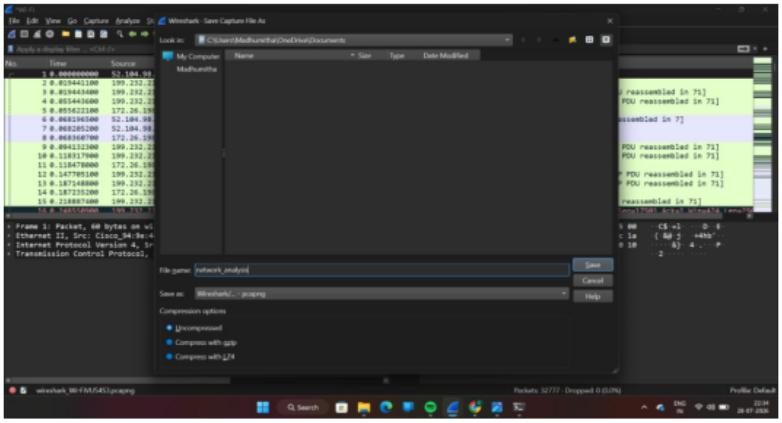


Evidence 13

Analyzing endpoints and conversations using Wireshark Statistics.

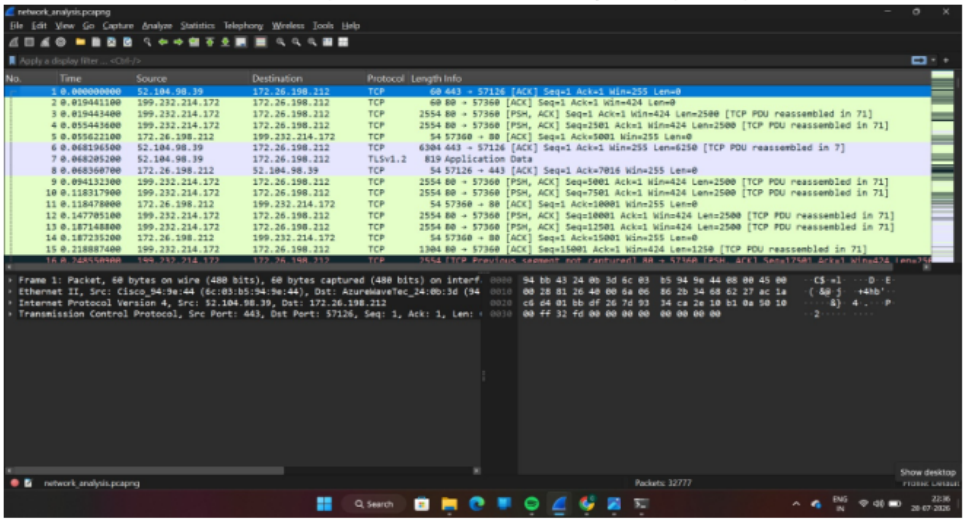
Step 21: Save Capture File

Description:
The captured network traffic was saved as a .pcapng file for future analysis.



Step 22: Open Saved Capture File

Description:
The saved PCAPNG file was reopened in Wireshark to verify the captured packet data.

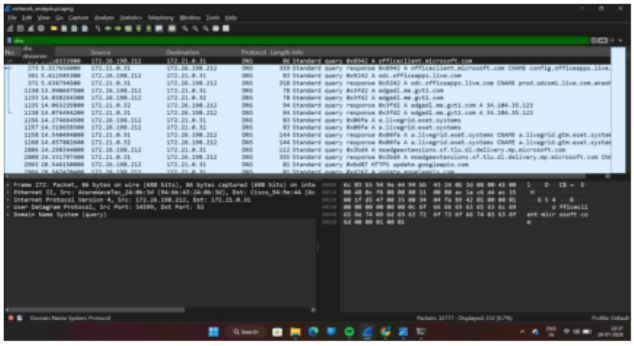


Evidence 14

Saving the capture as PCAPNG and reopening the saved file.

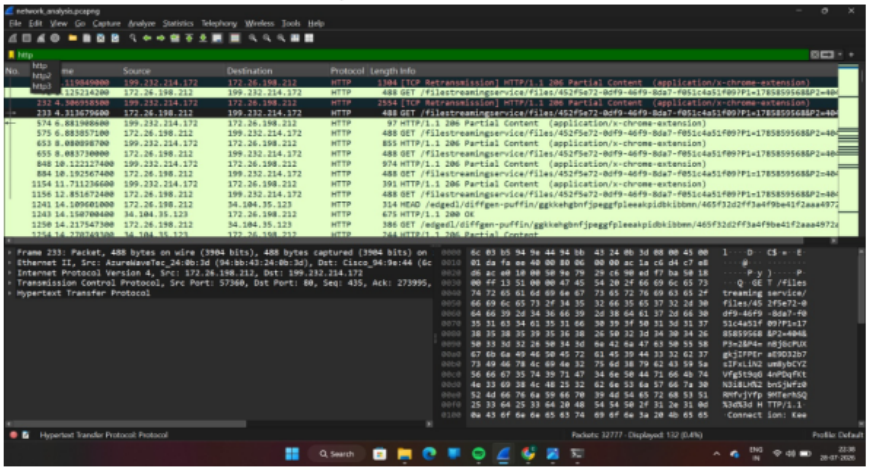
Step 23: Final Traffic Analysis

Description:
The filtered packets and packet details were reviewed to summarize HTTP, DNS, and ICMP traffic analysis.



Step 24: Final Result

Description:
The final Wireshark screen showed successful packet capture, protocol filtering, DNS query identification, and IP address analysis.



Evidence 15

Final traffic analysis and successful completion of the project.

Result:

Network traffic was successfully captured and analyzed using Wireshark. HTTP, DNS, and ICMP packets were filtered, DNS queries were identified, and IP addresses were examined for suspicious communication.

GitHub README Content

Repository name: Network-Traffic-Analysis-Wireshark

Suggested repository structure:

Network-Traffic-Analysis-Wireshark/
■■■ README.md
■■■ screenshots/
■■■ report/
  ■■■ Network_Traffic_Analysis_GitHub_Project.pdf

README description:

A practical cybersecurity project demonstrating network packet capture and analysis using Wireshark. The project covers HTTP, DNS, and ICMP traffic analysis, IP address review, endpoint/conversation analysis, and saving packet captures for further investigation.

Result:

Network traffic was successfully captured and analyzed using Wireshark. HTTP, DNS, and ICMP packets were filtered, DNS queries were identified, and IP addresses were examined for suspicious communication.

Note: This report is formatted as a project document based on the screenshots and descriptions in the supplied report. It does not claim malware was detected; the supplied work demonstrates traffic analysis and review for potentially unusual communication.